



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2026 - HM



Chapitre 6 : Archive JAR & Javadoc

1. Introduction aux archives JAR
2. Structure d'une archive JAR
3. Création d'une archive JAR
4. Fichier *Manifest*
5. Créer des exécutables (.exe pour Windows)
6. Générateur de documentation : *javadoc*

Ensicaen©2025 - HM

2

I. Introduction aux archives JAR

- Une archive JAR (**J**ava **AR**chive) est un format de fichier utilisé pour regrouper plusieurs fichiers Java (classes, ressources (images, icônes), métadonnées,...) dans un seul fichier compressé (format Zip).
- Les archives JAR simplifient la distribution des applications Java.
- Les archives JAR permettent de regrouper toutes les dépendances d'une application (Toutes les classes sont disponibles lors de l'exécution de l'application).
- Bonne pratique : pensez à importer seulement les classes utilisées.
- Insérer des informations additionnelles (version, auteur,...)

II. Structure d'une archive JAR

- Une archive JAR est un fichier compressé qui contient une structure de répertoires et de fichiers.
 - **Répertoires** : Respectent la structure des packages utilisés dans le projet Java.
 - **Fichiers .class** : compilation des fichiers .java
 - **Ressources** : Les fichiers de ressources (images, fichiers de configuration, etc.) sont généralement stockés dans des répertoires, comme ressources/ ou config/
 - **Fichier manifeste (MANIFEST.MF)** : situé dans le répertoire **META-INF/** de l'archive JAR. Il contient des métadonnées sur l'archive (version, auteur,...) et la classe principale (**Main-Class**) à exécuter.

III. Création d'une archive JAR

- La commande **jar** est un outil en ligne de commande fourni avec le JDK pour créer, manipuler et extraire des archives JAR.
- Syntaxe : `jar [options] [fichier-jar] [fichiers-ou-repertoires]`
- Options utilisées avec la commande jar :
 - **c** : Crée une nouvelle archive JAR.
 - **v** : Affiche des informations détaillées (verbose)
 - **f** : Spécifie le nom du fichier JAR à créer ou à manipuler.
 - **m** : Inclut un fichier manifeste (contenant des informations comme la classe principale) dans l'archive JAR.
 - **x** : Extrait les fichiers d'une archive JAR.
 - **t** : Liste le contenu d'une archive JAR.

III. Création d'une archive JAR

Commande	Opération
<code>jar cfm fichier.jar manifest.txt Main.class</code>	Crée une archive du Main avec un fichier manifest
<code>jar tf fichier.jar</code>	Afficher le contenu du fichier Jar
<code>jar xf fichier.jar</code>	Extraire le contenu du fichier Jar
<code>java -jar fichier.jar</code>	Lancer l'application Java depuis le fichier Jar

III. Création d'une archive JAR

- Soit le répertoire mon-projet contenant les fichiers suivants :

↳ java2025/

↳ HelloWorld.java

↳ HelloWorld.class

↳ manifest.txt

Le fichier **manifest.txt** doit contenir :

Main-Class: HelloWorld // Classe avec la méthode main()

→ Laisser une ligne vide à la fin du fichier manifest.txt

Pour créer le JAR :

```
jar cfm HelloWorld.jar manifest.txt HelloWorld.class
```

Pour exécuter le JAR :

```
java -jar HelloWorld.jar
```

Si le programme Java contient plusieurs classes, alors il faut regrouper les classes dans un dossier avec l'option **-C out**

```
↳ jar cfm fichier.jar manifest.txt -C out .
```

IV. Fichier Manifest

- Dans le fichier manifest.txt, vous pouvez ajouter plusieurs types d'informations :
 - **Main-Class:** Main ↳ Indique nom de la classe exécutable.
 - **Class-Path:** lib/dependance1.jar lib/dependance2.jar ↳ fichiers JAR nécessaires au fonctionnement de l'application.
 - **Implementation-Title:** MonApplication
 - **Implementation-Version:** 1.0.0
 - **Implementation-Vendor:** MonEntreprise ↳ informations utiles pour identifier votre application
 - **Built-By:** NomDuDéveloppeur
 - **Build-Jdk:** 17 ↳ indiquer la version du JDK utilisée et l'auteur du build.

V. Créer des exécutables (.exe pour Windows)

Il existe plusieurs logiciels gratuits qui permettent de créer des exécutables (.exe pour Windows) à partir de fichiers JAR.



<https://sourceforge.net/projects/launch4j/>



<https://jsmooth.sourceforge.net/>



<https://docs.oracle.com/en/java/javase/17/docs/specs/man/jpackage.html>



VI. Générateur de documentation : javadoc

- Par défaut, la documentation de java générée est au format HTML.
- Toute la documentation des API java a été générée grâce à **javadoc**
- Accéder à la documentation officielle de java :

<https://docs.oracle.com/en/java/javase/17/docs/api/index.html>
- Les commentaires Javadoc commencent par **/**** et se terminent par ***/**.
- Ils doivent être placés juste avant l'élément qu'ils documentent.
- javadoc utilise des **"tags"** pour préciser le rôle de certains composants d'un élément

VI. Générateur de documentation : javadoc

Classes	@author	Indique l'auteur du programme
	@version	Indique la version du Build
	@since	Date d'existence de ce programme
	@see	Spécifie un élément en relation avec l'élément documenté
Méthodes	@return	Indique ce que la méthode retourne
	@param	Documente un paramètre d'une méthode
	@deprecated	Indique qu'une méthode, une classe ou un champ est obsolète
	@exception	Spécifie qu'une exception peut être levée
	@throws	Similaire à @exception
Autres	@link	Insérer un lien vers une autre classe, méthode
	@value	Injecter une constante de classe dans le texte de la Javadoc.
	@serial	Documenter les champs qui sont sérialisés